

Experiment No.:	2
Experiment Title:	Email Spam/Ham Classification using Naive Bayes and KNN
Name:	NAVINKUMAR S
RegnNo:	3122247001039
Sem and AcademicYear:	Vth sem (2026-2027)
Submission Date	02-08-2026

1 Objective

The objective of this experiment is to develop a system that can tell whether an email is spam or ham (not spam), using two types of machine learning models: Naïve Bayes and K-Nearest Neighbours (KNN). Along with building these models, we compare the three versions of Naïve Bayes (Gaussian, Multinomial and Bernoulli), studies how changing the value of k affects KNN, tunes KNN using GridSearchCV and RandomizedSearchCV, compares two different ways of searching for neighbours (KDTree and BallTree), and checks how much time each model takes to train and predict. The models are also tested using 5-fold cross validation to make sure the results are reliable and not just a result of one lucky train-test split.

2 Problem Statement

Email spam is a common problem where unwanted or harmful emails are sent in bulk to users. The goal here is to automatically separate spam emails from normal (ham) emails using only numeric features taken from the email content, such as how often certain words or characters appear.

- **Input:** 57 numeric features describing each email (word frequencies, character frequencies, and statistics about capital letters).
- **Output:** A label telling whether the email is spam (1) or ham (0).
- **Objective:** Train classification models that can predict this label correctly for new, unseen emails, and compare which model and settings work best.

3 Dataset Description

Dataset Name	Spambase Dataset
Dataset Source	Kaggle – https://www.kaggle.com/datasets/somesh24/spambase
Number of Samples	4601 (original), 4210 after removing duplicate rows
Number of Features	57 numeric features
Number of Classes	2 (Spam = 1, Ham = 0)
Missing Values	None

Train-Test Split

80% train (3368 rows), 20% test (842 rows),
stratified on the class label

Out of the original 4601 emails, 2788 are ham and 1813 are spam, so the dataset is not perfectly balanced but is not extremely skewed either. After removing 391 duplicate rows, 4210 rows were used for the rest of the experiment.

4 Exploratory Data Analysis

Before training any model, the dataset was explored to understand its structure, check for missing or duplicate data, and see how the features and the target class are spread out. All the plots below were generated in the notebook.

Missing Value Check

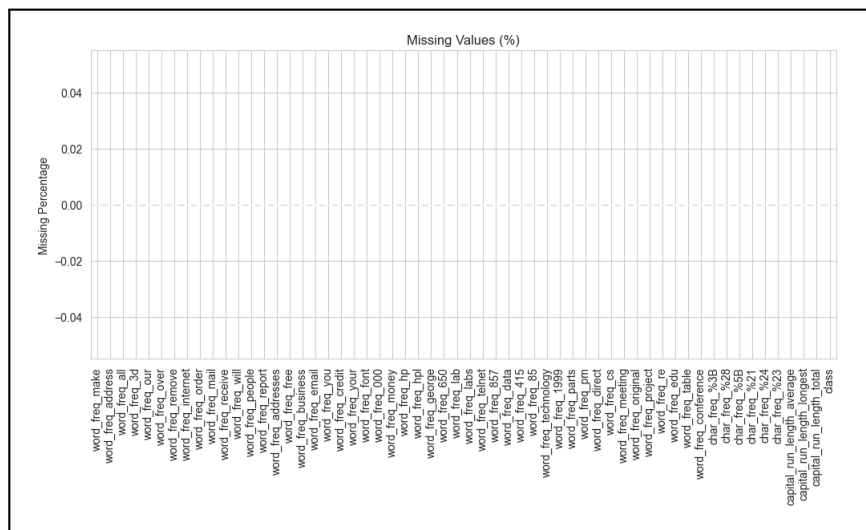


Figure 1: Missing Values Bar Chart

Inference:

The dataset has no missing values in any of the 57 features or the target column. Thus no rows needed to be dropped or imputed for filling the missing values. Therefore the preprocessing work should focus only on scaling and removing duplicate rows instead of handling missing data.

Duplicate Rows

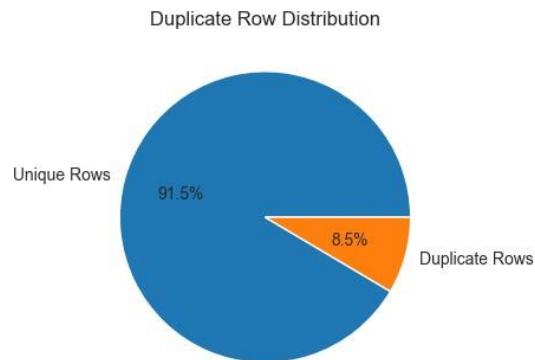


Figure 2: Duplicate Row Distribution Pie Chart

Inference:

Out of 4601 rows, 391 rows were exact duplicates of other rows. These were removed so that the same email record does not appear in both the training and test sets. After removing duplicates, the dataset contains 4210 rows (out of which 2531 rows are spam emails and 1679 are ham emails).

Class Distribution

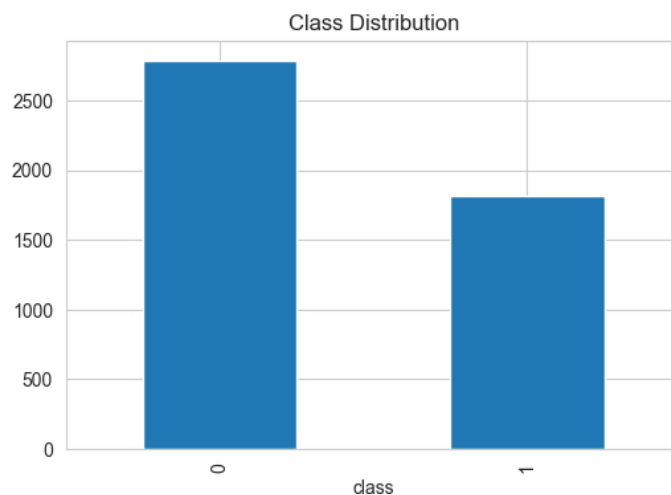


Figure 3: Class Distribution Bar Chart – count of Spam (1) vs Ham (0) emails

Inference:

There are more ham emails than spam emails in the dataset, like 60% ham and 40% spam. The imbalance is mild, so considering accuracy alone is reasonably fair metric, but precision, recall and F1-score are still reported for a more complete picture, since they are more sensitive to how well the smaller (spam) class is being predicted.

Correlation Matrix

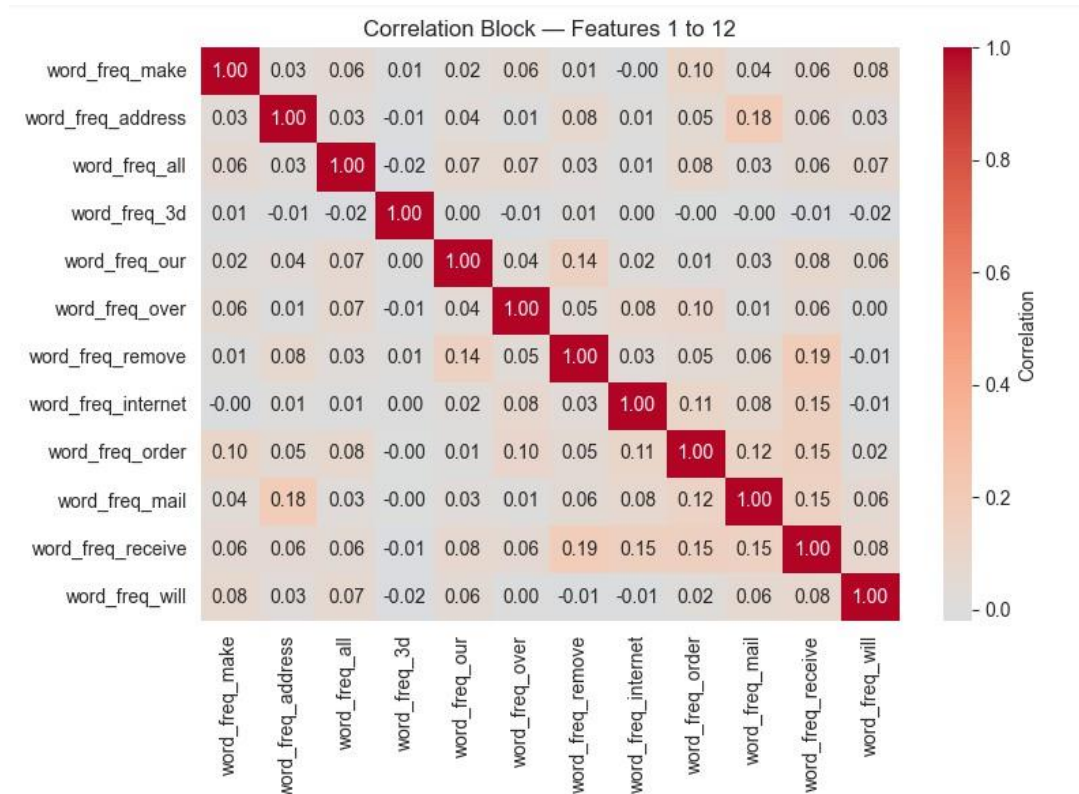


Figure 4: Correlation Heatmap of the 57 numeric features (from the notebook, Step 4, correlation_analysis output)

Inference:

The heatmap shows how strongly each pair of features is related to each other. Most word-frequency features are only weakly related to one another, since each word appears somewhat independently. A few features, such as the different capital-letter statistics, are more strongly correlated with each other because they measure related aspects of the same thing (use of capital letters). No pair of features was so strongly correlated that it needed to be removed before training.

Feature Distribution – Histogram

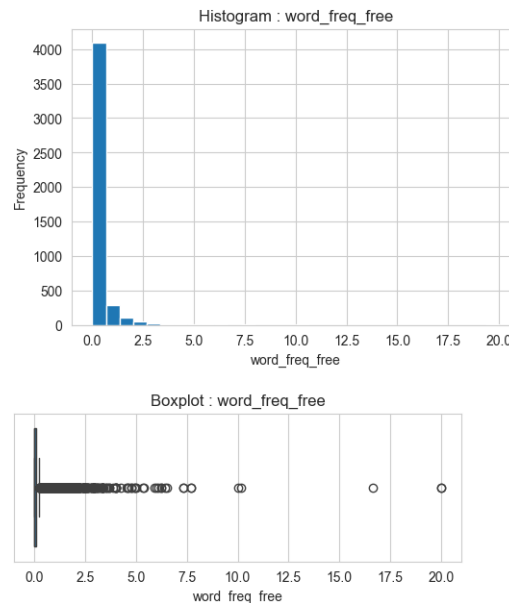


Figure 5: Histogram of an word_freq_free

Inference:

Most of the word-frequency and character-frequency features have a large number of zero values, since a given word does not appear in most emails, with a long tail of higher values for the emails where that word does appear often. This kind of skewed shape is one of the reasons Gaussian Naïve Bayes (which assumes a bell-curve shape) does not fit this dataset perfectly, while Multinomial and Bernoulli Naïve Bayes, which are built for count-like or presence/absence data, are more natural fit.

Feature Distribution – Boxplot

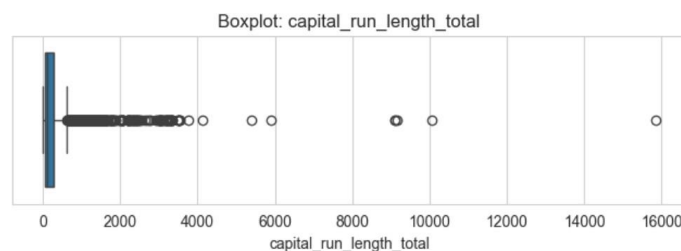


Figure 6: Boxplot of ancapiatal_run_length_total

Inference:

The boxplots show that several features have a good number of outliers – emails where a word or character appears far more often than usual. These are not necessarily errors; spam emails often do use certain words or capital letters much more heavily than normal emails, so these outliers can actually be useful for telling spam and ham apart rather than noise that needs to be removed.

5 Data Preprocessing

- **Missing value handling:** The dataset had no missing values, so methods like filling in or dropping of values was required.
- **Encoding:** Not needed, since all 57 features and the target class are already numeric.
- **Duplicate removal:** 391 duplicate rows were removed before splitting the data, so the model is not trained and tested on the same repeated record.
- **Feature scaling:** Naive Bayes models were trained on the features in their original scale (Gaussian and Multinomial NB used the raw frequency values, Bernoulli NB used a 0/1 version). KNN, on the other hand, measures distance between points, so its features were scaled using StandardScaler before training, fitting the scaler only on the training data to avoid leaking test data information into training.
- **Train-test split:** An 80/20 split was used, stratified on the class label so that the ratio of spam to ham stays the same in both the training data set and the test data set.
- **Feature engineering:** No new features were created. The 57 original features from the dataset were used as they are.

6 Machine Learning Algorithm

Naïve Bayes

Working principle: Naïve Bayes is a probabilistic classifier based on Bayes' theorem with conditional independence assumptions.

Mathematical formulation:

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)}$$

Here C is the class (spam or ham) and X is the set of feature values for an email. $P(C)$ is how common that class is overall, $P(X | C)$ is how likely those feature values are for that class, and $P(C | X)$ is what we want to find: the probability of class given the email's features.

Three versions were used in this experiment:

- **Gaussian NB** assumes each feature follows a normal (bell-curve) distribution within each class.
- **Multinomial NB** assumes the features behave like counts or frequencies, which fits the word-frequency style features in this dataset.
- **Bernoulli NB** assumes the features are binary (present or not present), so the features were converted to 0/1 before using this model.

Advantages: Very fast to train and predict, works well even with a small amount of data.

Limitations: The independence assumption can hurt accuracy when features are actually related to each other.

K-Nearest Neighbours (KNN)

Working principle: KNN is a lazy learning algorithm. Instead of learning a fixed rule during training, it simply stores the training data. When a new email comes for classification, it looks at the k closest training examples (based on distance, which is a hyperparameter) and assigns the class that is most common among those neighbours.

Mathematical formulation: Euclidean distance:

$$d(X_1, X_2) = \sqrt{\sum_{i=1}^n (x_{1i} - x_{2i})^2}$$

or the Manhattan distance:

$$d(X_1, X_2) = \sum_{i=1}^n |x_{1i} - x_{2i}|$$

Both distance measures were tried in this experiment as part of hyperparameter tuning.

Advantages: Simple to understand, does not assume anything about how the data is distributed, and can work well when similar emails really do have similar labels.

Limitations: Prediction is slow because it has to compare a new point against many stored training points (or search a tree built over them when KDTree or BallTree are used). It is also sensitive to the scale of the features, so scaling is required, and it can be affected by noisy features.

7 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook
Operating System	Ubuntu 22.04 LTS
Hardware	Acer Aspire Lite (Intel Core i5-12450H, 16 GB RAM, 512 GB SSD)

8 Hyperparameter Selection

The KNN hyperparameters were tuned using both GridSearchCV and RandomizedSearchCV, searching over the same set of options with 5-fold cross validation, optimizing the F1-score. Both methods agreed on the same best settings, shown below.

Parameter	Value
n_neighbors (k)	9
weights	distance
metric	manhattan
algorithm	kd_tree (GridSearchCV) / ball_tree (RandomizedSearchCV)
leaf_size	20 (GridSearchCV) / 30 (RandomizedSearchCV)
Best CV F1-score	0.886

GridSearchCV vs RandomizedSearchCV: Both methods found the exact same best combination of k , weighting, and distance metric, and reached the same cross-validation

F1-score of 0.886. GridSearchCV checked every possible combination in the search space and took about 134 seconds, while RandomizedSearchCV checked only 30 randomly chosen combinations and took about 21 seconds – roughly 6 times faster, while still finding an equally good result. This shows that RandomizedSearchCV can be a good time-saving choice when the search space is large. But it is not sure that the model is optimally best among all the combinations of the parameters given, since it randomly chooses n (here 30) combinations, it will be one of the good models, but not the top performing model.

9 Results

The table below shows the performance of the final tuned KNN model (the best model overall) on the test set.

Metric	Value
Accuracy	0.9276
Precision	0.9599
Recall	0.8542
F1-score	0.9039
ROC-AUC	0.9746

Naïve Bayes Comparison

Metric	Gaussian	Multinomial	Bernoulli
Accuracy	0.8361	0.7886	0.8943
Precision	0.7240	0.7616	0.8777
Recall	0.9524	0.6845	0.8542
F1-score	0.8226	0.7210	0.8658
ROC-AUC	0.9525	0.8342	0.9609

Among the three Naïve Bayes versions, Bernoulli NB gave the best overall balance of precision, recall and F1-score, since many of the features in this dataset behave like presence/absence signals. Gaussian NB has the highest recall but a much lower precision, meaning it catches most spam but also wrongly flags a lot of ham emails as spam. Multinomial NB has performed on average.

KNN Comparison (varying k)

k	Accuracy	Precision	Recall	F1
1	0.9050	0.8832	0.8780	0.8806
3	0.9097	0.9063	0.8631	0.8841
5	0.9133	0.9148	0.8631	0.8882
7	0.9050	0.9103	0.8452	0.8765
9	0.9074	0.9243	0.8363	0.8781
11	0.9086	0.9164	0.8482	0.8810

Accuracy and F1-score change only slightly as k increases from 1 to 11, with $k = 5$ giving the best F1-score among these untuned values. After proper tuning, $k = 9$

combined with distance-based weighting and the Manhattan distance gave the best cross-validated result.

KDTree vs BallTree

Metric	KDTree	BallTree
Accuracy	0.9074	0.9074
Training Time (s)	0.0124	0.0082
Prediction Time (s)	0.1813	0.1709

Both KDTree and BallTree give exactly the same accuracy, since they only change how neighbours are searched for, not which neighbours are actually found. In this experiment, BallTree was slightly faster than KDTree for both training and prediction on this 57-feature dataset.

5-Fold Cross Validation

Fold	Naïve Bayes Best (Bernoulli)	KNN
1	0.9097	0.8729
2	0.9026	0.9074
3	0.9074	0.9299
4	0.9145	0.9192
5	0.7791	0.8468
Average	0.8827	0.8952

Across all 5 folds, the tuned KNN model gave a slightly higher and more stable average accuracy (0.895) compared to the best Naïve Bayes model (0.883). Bernoulli NB also had one fold (fold 5) where accuracy dropped noticeably.

10 Result Visualization

Confusion Matrix

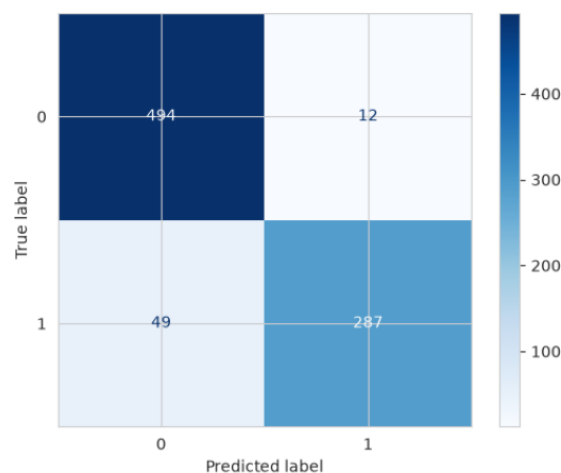


Figure 7: Confusion Matrix of the tuned KNN model on the test set

Inference:

The confusion matrix shows how many test emails were correctly and incorrectly classified. Most ham emails are correctly identified as ham, and most spam emails are correctly identified as spam. The main mistakes are a number of spam emails being missed and classified as ham (false negatives), that is about 15% of spams are undetected, while very few hams are wrongly marked as spam.

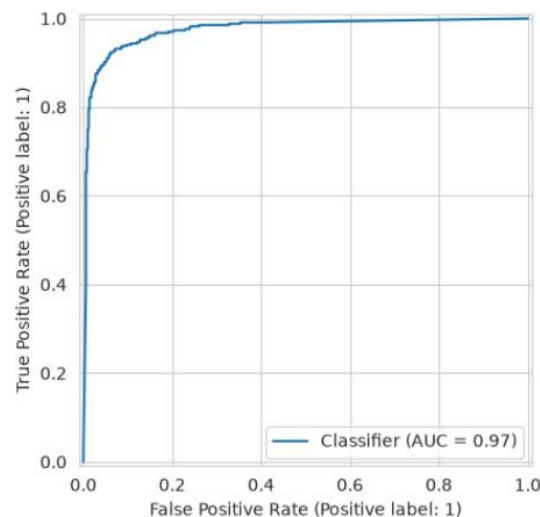
ROC Curve

Figure 8: ROC Curve of the tuned KNN model

Inference:

The ROC curve plots the true positive rate against the false positive rate at different thresholds. The area under this curve (ROC-AUC) is 0.9746, which is close to 1, showing that the model is very good at ranking spam emails higher than ham emails.

Precision-Recall Curves (All Models)

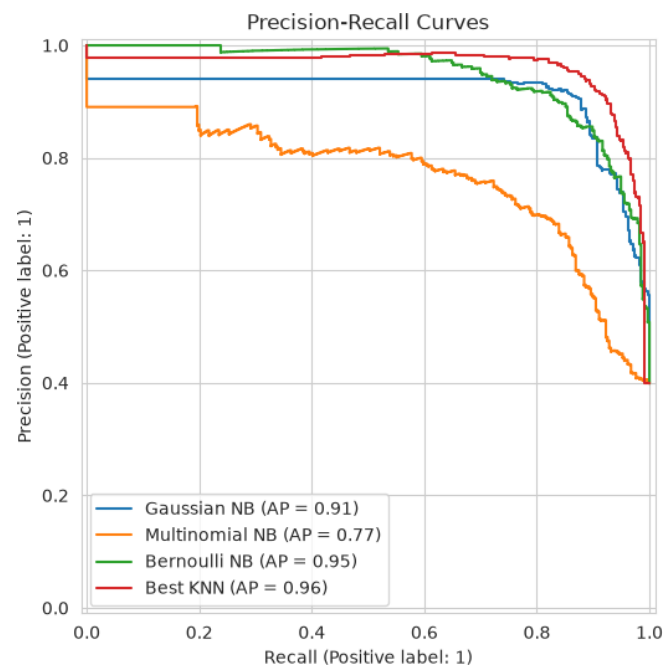


Figure 9: Precision-Recall Curves for Gaussian NB, Multinomial NB, Bernoulli NB and the tuned KNN model

Inference:

The tuned KNN curve stays closer to the top-right corner (high precision and high recall at the same time) compared to the Naïve Bayes curves, which confirms that KNN gives the best overall balance between catching spam and avoiding false alarms.

Accuracy vs k

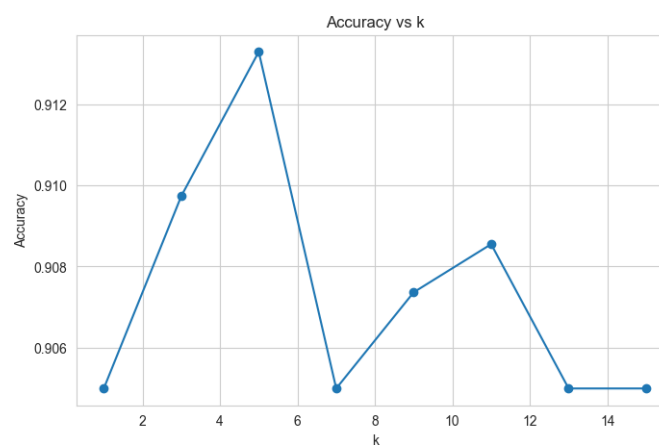


Figure 10: Accuracy vs k for KNN, k = 1 to 15

Inference:

Accuracy is high even at small values of k , changes only slightly as k increases, and starts to drop slowly once k becomes very large. This matches the general behaviour of KNN: very small k can overfit to noise, while very large k smooths the decision boundary so much that it starts underfitting. The best region is somewhere in the middle, which is confirmed by GridSearchCV picking $k = 9$ which was at $k=5$ in default KNN (Brute Force).

GridSearchCV Heatmap

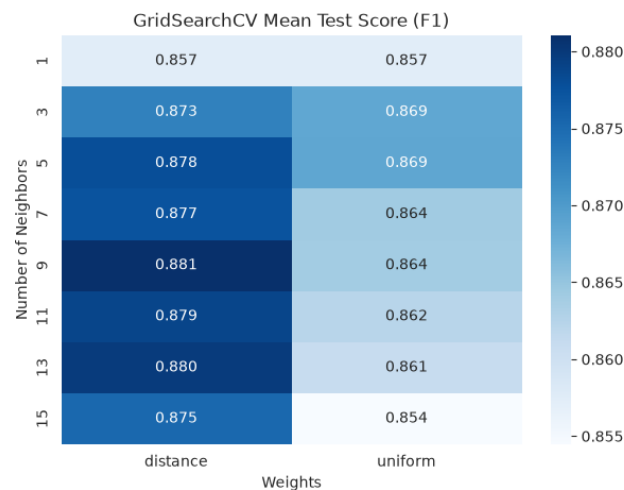


Figure 11: GridSearchCV mean F1-score for different values of k

Inference:

The heatmap shows that distance-based weighting generally scores a little higher than uniform weighting across most values of k . This makes sense because distance weighting gives closer neighbours more influence on the prediction than farther ones, which helps when the nearest few neighbours are more reliable than the rest.

RandomizedSearchCV Score Distribution

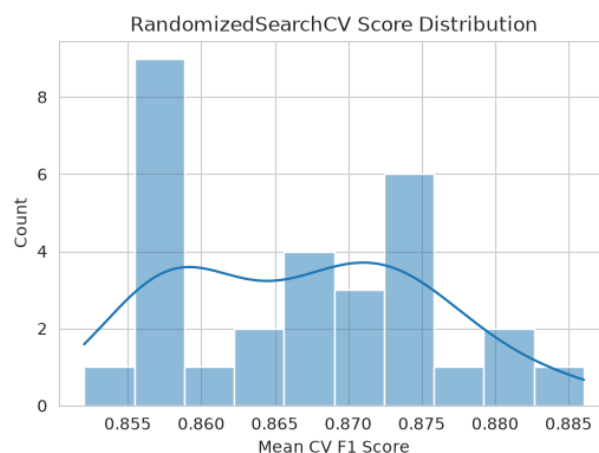


Figure 12: Distribution of mean cross-validation F1 scores

Inference:

Most of the randomly sampled parameter combinations scored reasonably close to each other, with a smaller number of combinations scoring lower. This shows that KNN's performance on this dataset is fairly stable across many different hyperparameter choices, and only a few combinations (for example, very small or very large k) perform worse.

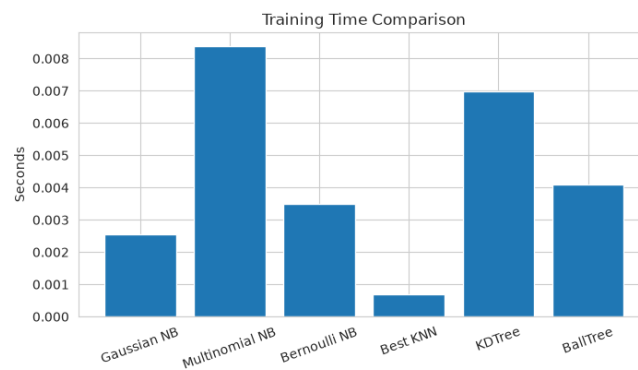
Training and Prediction Time Comparison

Figure 13: Training Time Comparison across Gaussian NB, Multinomial NB, Bernoulli NB, KNN, KDTree and BallTree

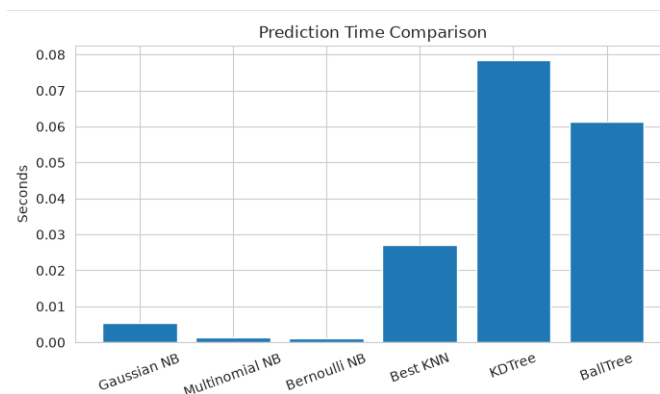


Figure 14: Prediction Time Comparison across the same models

Inference:

All three Naïve Bayes models train and predict in a few milliseconds, since they only need to calculate simple probability statistics from the data. KNN training is almost instant (it just stores the data), but its prediction step is clearly the slowest of all the models, since it has to search through the training data for every single prediction. This matches the theoretical time complexity, where Naïve Bayes scales much better than KNN as the dataset grows.

Classifier Comparison

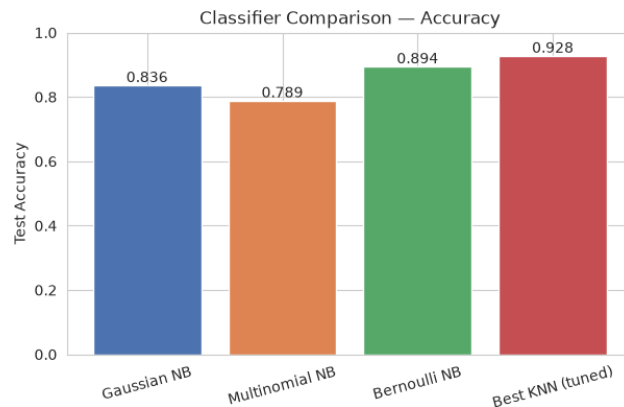


Figure 15: Overall Test Accuracy Comparison – Gaussian NB, Multinomial NB, Bernoulli NB and tuned KNN

Inference:

The tuned KNN model has the highest test accuracy among all four models, followed by Bernoulli NB, then Gaussian NB, and Multinomial NB has the lowest accuracy. Thus KNN (after tuning) is the recommended model for this dataset if prediction speed is not a concern.

Cross-Validation Accuracy

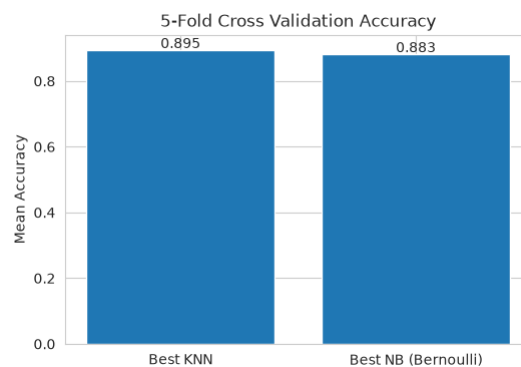


Figure 16: 5-Fold Cross Validation Mean Accuracy – Best Naïve Bayes (Bernoulli) vs Best KNN

Inference:

KNN's average cross-validation accuracy (0.895) is a little higher than Bernoulli NB's (0.883), and this pattern holds across most of the 5 folds, confirming that the difference seen on the single train-test split earlier was not just luck. Bernoulli NB does have one weaker fold, suggesting it may be a little more sensitive to which rows end up in the training set.

11 Discussion

Across all the experiments, KNN (after fine tuning with GridSearchCV) gave the best overall performance, with a test accuracy of 92.8% and an F1-score of 0.904, ahead of the best Naïve Bayes model (Bernoulli NB) which reached an accuracy of 89.4% and an F1-score of 0.866. Among the Naïve Bayes variants, Bernoulli NB worked best, most likely because many of the features behave like presence/absence signals once thresholded, which matches Bernoulli NB's assumptions closely. Gaussian NB struggled a little more because the word-frequency features are heavily skewed with many zeros, which does not match the normal-distribution assumption it relies on.

For KNN, distance-weighted voting and the Manhattan distance worked better than uniform weighting and Euclidean distance, and a moderate value of $k = 9$ gave the best cross-validated score – small enough to stay sensitive to genuinely different emails, but large enough to avoid being affected by single noisy neighbour. GridSearchCV and RandomizedSearchCV reached the exact same best results, but RandomizedSearchCV did so about 6 times faster, which is a useful trade-off when we need faster outputs. KDTree and BallTree gave identical accuracy, as expected, with BallTree slightly faster on this dataset.

The main limitation of KNN is its prediction speed, which grows with the number of stored training emails – this could become a real problem if the dataset were much larger or if predictions need to happen instantly. In contrast, Naïve Bayes, stays fast at both training and prediction regardless of dataset size, which would make it a better choice for very large-scale spam filtering, even though its accuracy on this dataset was lower compared to KNN. A possible improvement for future work to test feature selection to remove less useful features and speed up KNN's distance calculations.

12 Conclusion

In this experiment, developed and compared spam-detection models using Naïve Bayes (Gaussian, Multinomial, Bernoulli) and KNN on the Spambase dataset. Bernoulli NB was the best-performing Naïve Bayes variant, and KNN, once tuned with GridSearchCV/RandomizedSearchCV to $k = 9$, distance weighting and the Manhattan distance, gave the best result overall (accuracy 92.8%, F1-score 0.904, ROC-AUC 0.975). KDTree and BallTree performed identically in accuracy, with only small differences in speed. Cross-validation confirmed that KNN's advantage over Naïve Bayes was consistent and not just a result of one particular train-test split. For datasets of this size, KNN is the more accurate choice, while Naïve Bayes remains good choice for very large datasets.

13 References

1. Scikit-learn Developers, "scikit-learn: Machine Learning in Python," <https://scikit-learn.org/stable/documentation.html>.
2. Somesh, "Spambase Dataset," Kaggle, <https://www.kaggle.com/datasets/somesh24/spambase>.